

Comparative Analysis of User-Defined and Built Functions: Accuracy, Efficiency, and Performance

Goal of Experiment

This experiment seeks to determine the efficiency and accuracy of user-defined mathematical functions against C's mathematical library <math.h>.

The following functions were tested: exponentiation, square root, natural log, sine, cosine, tangent, and factorial. The points of interest are differences in times of execution and differences in accuracy.

Findings and Discussion

1. Exponentiation (x^y)

- **Findings:** The user-defined exponentiation function produced the same results as the library `pow( )` function; no difference in accuracy.
- **Efficiency:** The time of execution of the built-in function was less than the user-defined; however, it was a marginal difference.

Conclusion: The power function is a success.

2. Square Root

- ✓ **Findings:** The user-defined square root function produced the same results as that of the library `sqrt( )` function.
- ✓ **Efficiency:** Execution time was instantaneous. Time of execution of built-in was less than the user-defined; this is a positive reinforcement for using library functions over user-defined ones for basic operational use.

Conclusion: The square root function is a success.

3. Logarithm

- ✧ **Findings:** User-defined logarithm function produces expected output.
- ✧ **Efficiency:** Execution time is slightly longer than built-in `log()`. But still efficient.

Conclusion: The natural logarithm function is a success.

4. Trig (Sin, Cos, Tan)

Findings:

- ❖ The sine and cosine functions returned values quite identical to built-in functions of sine and cosine, which means these user-defined functions worked with just non-significant levels of accuracy difference.
- ❖ For user-defined tan, when $\tan(x)$ is undefined (i.e., when $\cos(x) = 0$), my function correctly identified cases where $\tan(x)$ is undefined due to $\cos(x)=0$, preventing computation errors. The built-in $\tan()$ function doesn't return infinity but instead produces a very large number.

Efficiency: Execution time of user-defined is slightly longer than built-in equivalents but still efficient.

Conclusion: The trigonometric functions are a success.

5. Factorial Function

- **Findings:** The factorial returned the correct results. There is no direct built-in factorial function relative to `<math.h>`, thus making this all the more important.
- **Efficiency:** Execution time was negligible.

Conclusion: The factorial function is a success.

Key Insights & Conclusion

1. **Correctness:** The user-defined functions successfully replicate built-in functions with good accuracy.
2. **Speed:** Built-in functions were always better because they were faster, always optimized for speed, and ran faster in all cases.
3. **Error Handling:** The user-defined tangent function handled the undefined values, allowing it to compute properly.
4. **Efficiency:** User-defined functions provide a greater understanding of how and why manual math calculations happen; however, for effective and efficient purposes and optimization for large efforts, built-in functions rule.

Conclusion

User-defined functions are useful for understanding and practicing the methods and mathematics that go on behind the scenes e.g., educational and learning purposes, but for ultimate practicality and limited execution time e.g., real-world applications demanding high efficiency and minimal execution time, the built-in, readily available, and accessible functions are the way to go.

In the end, **the built-in functions are faster and recommended, but the user-defined functions provide a level of transparency and reasoning behind later computations.**